

HomeOffice Assessment Test Automation Framework Overview

A detailed walkthrough of the Java Serenity BDD, Cucumber, Selenium, REST Assured, and WireMock test framework.

1. Big Picture

This repository is a Java test automation framework that covers both UI and API testing. UI tests target SauceDemo, while API tests target Postcodes.io. The framework is written around Cucumber feature files, Serenity BDD reporting and lifecycle support, Selenium WebDriver for browser automation, REST Assured for HTTP testing, and WireMock for deterministic API stubbing.

Layer	Technology	Purpose
Runner	JUnit Platform Suite	Starts Cucumber from Maven/JUnit.
BDD	Cucumber 7	Maps Gherkin scenarios to Java step definitions.
UI automation	Serenity BDD + Selenium	Controls SauceDemo in a browser and records rich reports.
API automation	Serenity REST + REST Assured	Sends HTTP requests and validates Postcodes.io responses.
Stubbing	WireMock	Runs deterministic API scenarios without relying on the live service.
Assertions	AssertJ + JSON schema validator	Provides readable validation for responses, UI state, and schemas.

2. Project Layout

src/test/java/homeoffice	
actions/	UI workflow/action classes
api/	Postcodes.io REST client and request setup
assertions/	Reusable API assertions
config/	API configuration loader and validator
constants/	Runner/report constants
model/	Small value objects such as Coordinates
PageObject/	Serenity page objects for SauceDemo
pagecollections/	Page alias registry and page resolution
state/	API scenario state holder
StepsDefinitions/	Cucumber step definitions
support/	Hooks, WireMock stubs, browser logging helpers

```
src/test/resources
  features/api_tests   Postcodes.io feature files
  features/ui_tests    SauceDemo feature files
  schemas/             JSON schemas
  stubs/               WireMock mappings and response bodies
  serenity.conf        Serenity, WebDriver, and API configuration
```

3. How Test Execution Starts

The main suite is `src/test/java/homeoffice/CucumberTestSuite.java`. It uses the Cucumber JUnit Platform engine, selects the feature root `/features`, and sets the glue package to `homeoffice`. That means Cucumber scans all feature files and discovers step definitions and hooks anywhere below the `homeoffice` package.

```
@Suite
@IncludeEngines("cucumber")
@SelectClasspathResource("/features")
@ConfigurationParameter(key = GLUE_PROPERTY_NAME, value = "homeoffice")
@ConfigurationParameter(key = PLUGIN_PROPERTY_NAME, value = DEFAULT_PLUGINS)
```

There is also a UI-specific runner at `src/test/java/homeoffice/testRunners/CucumberTestSuite.java`. It points at `/features/ui_tests`, so it can run only the browser-focused scenarios.

4. Feature Layer

Feature files are the behaviour specification layer. They describe what the user or API consumer expects, not how Selenium or REST Assured should do it.

API Feature Example

```
@postcodes-api @stub
Feature: Find the nearest postcode using coordinates from a postcode lookup

Background:
  Given the API root endpoint is reachable
  Given the "<postcode>" to search

Scenario Outline: Verify every field returned for a postcode lookup
  When I search postcode for "<postcode>"
  Then the response status should be "<code>"
```

UI Feature Example

```
@ui @login
Feature: login page UI

Scenario Outline: Login page displays logo, form, accepted users and password
```

```
Given I navigate to "/"
And the page title should be "Swag Labs"
Then I should be on the "login" page
And the login form should be present and visible
```

Tags drive execution and environment behaviour. For example, `@stub` starts WireMock and points API traffic at the stub server, while `@live` stops the stub and uses the real Postcodes.io base URL.

5. Step Definition Layer

Step definitions translate Gherkin into Java. They should stay thin: read input, update state, call an action/API method, or call an assertion helper.

The main API step class is `PostcodeStepDefinitions`. It owns a fresh `PostcodeScenarioState` per scenario and injects the `PostcodesApi` client through Serenity's `@Steps`.

```
@Before
public void initialiseScenarioState() {
    state = new PostcodeScenarioState();
}

@When("I request the postcode details")
public void iRequestThePostcodeDetails() {
    state.setLookupResponse(postcodesApi.lookupPostcode(state.getPostcodeLookoutFor()));
}
```

UI step definitions follow a similar pattern, but usually delegate to action classes. For example, navigation steps call `NavigationAction`, and login steps call login-focused action classes.

6. API Framework Flow

The API side follows this flow:

1. A Gherkin step describes an API operation.
2. The step definition stores inputs in `PostcodeScenarioState`.
3. The step calls `PostcodesApi`.
4. `PostcodesApi` builds the request using `PostcodesRequestSpecification`.
5. REST Assured sends the request to either Postcodes.io or WireMock.
6. The response is stored back into scenario state.
7. Assertion steps call `PostcodesAssertions`.

API Client

`PostcodesApi` wraps the HTTP operations into intent-based methods:

- `lookupPostcode(String postcode)`
- `nearestPostcodes(double longitude, double latitude)`

- `bulkLookupPostcodes(List<String> postcodes)`
- `nearestOutcodes(String outcode)`
- `get(String path)`

This prevents feature or step files from hard-coding request details repeatedly.

Request Specification

`PostcodesRequestSpecification` creates a common REST Assured request using the configured base URI, timeout, URL encoding, Serenity REST integration, and optional request/response logging.

API Configuration

`PostcodesApiConfiguration` reads values from system properties, Serenity environment variables, and `serenity.conf`. It centralizes paths, query parameter names, status codes, timeout values, JSON paths, and stub resource locations.

7. API Scenario State

`PostcodeScenarioState` holds scenario data such as the current postcode, bulk postcode list, lookup response, nearest response, current response, extracted coordinates, and nearest postcode.

The important design point is `currentResponse`. When a lookup response is set, it becomes the current response. When a nearest or bulk response is set, that becomes the current response. This lets generic assertions such as `Then the response status should be "200"` work after different request types.

8. API Assertions

`PostcodesAssertions` contains reusable assertion methods for:

- status code validation
- response time checks
- postcode normalization and matching
- JSON schema validation
- exact JSON resource comparison
- JSON path type and value validation
- coordinate extraction and comparison
- nearest-postcode distance ordering
- sensitive implementation detail leakage checks

9. WireMock Stubbing

WireMock support is handled by `TestEnvironmentHooks` and `PostcodesApiStub`.

Before all tests, the framework validates configuration. Before each `@stub` scenario, it starts WireMock on a dynamic port, loads stub mappings, and sets `postcodes.api.base-url` to the WireMock server URL. Before each `@live` scenario, it stops the stub and clears the override so the live API base URL is used.

Stub mappings come from `src/test/resources/stubs/postcodes-stub-mappings.csv`. Response bodies come from `src/test/resources/stubs/responses`.

```
method,pathPattern,queryParams,status,responseFile
GET,/,200,api-available.json
GET,/postcodes/.*,404,lookup-not-found.json
GET,(?i)/postcodes/SW1A(%20)?1AA,,200,lookup-success.json
POST,/postcodes,,200,bulk-postcode-lookup-success.json
```

10. UI Framework Flow

The UI side follows this flow:

1. A Gherkin step describes a browser action or assertion.
2. The step definition delegates to an action class.
3. The action class performs a workflow-level operation.
4. The page object performs low-level Selenium/Serenity interaction.
5. Assertions validate page state, URL, title, text, element visibility, or product/cart state.

```
Feature file
-> Step definition
  -> Action class
    -> Page object
      -> Serenity PageObject / Selenium WebDriver
```

BasePage

`BasePage` extends Serenity's `PageObject` and provides common browser helpers: waits, clicks, typing, scrolling, URL checks, title checks, element visibility checks, text reads, JavaScript fallback interactions, and action-based fallback interactions.

Specific Page Objects

Specific pages such as `LoginPage`, `InventoryPage`, `CartPage`, and checkout pages hold page-specific locators and operations. For example, `LoginPage` knows the username field, password field, login button, error message, accepted usernames panel, and inventory page validation.

Action Classes

Action classes express user workflows above the page-object level. For example, `LoginActions` resolves friendly aliases such as `standard user` to `standard_user`, applies the shared SauceDemo password, then calls the login page object.

11. Page Registry

`PageCollection` acts as a page alias registry. It maps friendly names such as `login`, `products`, `cart`, and `checkout overview` to URL fragments, ready locators, page objects, and aliases.

This allows readable feature steps such as:

```
Then I should be on the "cart" page
```

The framework resolves `cart` into the expected URL fragment and ready locator, then validates that the browser is on the right screen.

12. Reporting

Reporting plugins are configured in `RunnerConstants`. The framework can produce Serenity reports, Extent reports, Cucumber HTML, Cucumber JSON, Cucumber JUnit XML, and timeline reports.

The main Serenity report is expected at:

```
target/site/serenity/index.html
```

Additional Cucumber reports are configured under:

```
target/cucumber-reports/  
build/test-results/timeline/
```

13. End-to-End Example

For an API scenario tagged `@stub`:

1. Cucumber sees the `@stub` tag.
2. `TestEnvironmentHooks` starts WireMock.
3. WireMock sets `postcodes.api.base-url` to its dynamic URL.
4. The `Given` step stores the postcode in scenario state.
5. The `When` step calls `PostcodesApi.lookupPostcode`.
6. `PostcodesRequestSpecification` builds the REST Assured request.
7. The request goes to WireMock and receives a stub JSON response.
8. The response is saved into `PostcodeScenarioState`.
9. The `Then` step calls `PostcodesAssertions.assertStatus`.
10. Serenity and Cucumber write the result into the configured reports.

14. Main Strengths

- The framework has a clear layered structure from feature files to steps, actions/API clients, page objects, config, and assertions.
- The API side is deterministic when run with `@stub`, which avoids dependency on a public service for most checks.
- Configuration is centralized and supports system-property overrides.
- Assertions are centralized, readable, and reusable.
- The UI page registry makes page validation steps more readable in Gherkin.
- Serenity gives useful execution reports, screenshots, and REST evidence.

15. Recommended Improvements

Consolidate Older and Newer Step/Action Styles

The framework currently contains a mixture of direct step definitions and action-based step flows. Some scenarios call action classes through thin Cucumber step definitions, while other steps contain more workflow logic directly. Both approaches work, but mixing them makes the framework harder to scan, extend, and debug because similar behaviours can be implemented in different places.

A good direction would be to standardise the UI framework around this structure:

```
Feature file
-> Step definition
   -> Action class
       -> Page object
```

Step definitions should mainly translate Gherkin arguments and delegate. Action classes should hold reusable workflows, such as logging in, adding products to the cart, or completing checkout. Page objects should remain focused on page locators and page-level interactions. This would reduce duplicated logic and make it clearer where new behaviour belongs.

Remove Duplicate Step Annotation

The method that handles `Given the Postcodes.io API is available at "/"` currently has the same Cucumber annotation twice:

```
@Given("the Postcodes.io API is available at {string}")
@Given("the Postcodes.io API is available at {string}")
```

This should be reduced to a single annotation. The duplicate does not add any behaviour and can create unnecessary confusion when maintaining or searching step bindings. If the root path is always used, the step could also be renamed to something more specific, such as:

```
Given the Postcodes.io API root endpoint is available
```

That version would avoid passing `"/"` through every background step and would make the intent clearer.

Exclude Generated Output From Searches and Reviews

Generated folders such as `target`, `build`, and report directories contain compiled classes, generated reports, screenshots, logs, and copied test resources. They are useful execution outputs, but they add a lot of noise during code searches and reviews.

For day-to-day maintenance, searches should focus on source and resource folders such as `src/test/java` and `src/test/resources`. Generated folders should also be kept out of committed changes unless there is a deliberate reason to version a specific artifact. This keeps reviews smaller and prevents generated files from hiding meaningful changes.

Make API Scenario State Naming More Precise

The API scenario state should describe the type of response it is holding as accurately as possible. At the moment, the bulk postcode lookup flow stores its response in the nearest-response field. The tests still pass because the same setter also updates `currentResponse`, and the generic assertion steps read from `currentResponse`. However, the field name suggests the response came from a nearest-postcode request, which is misleading when debugging failures or adding new endpoint coverage.

A clearer model would keep `currentResponse` for shared assertions, but also store each endpoint response in a field with a specific name:

```
lookupResponse
bulkLookupResponse
nearestPostcodeResponse
nearestOutcodeResponse
currentResponse
```

Each setter can still update `currentResponse`, preserving the existing generic assertion style while making the state object accurately describe what each response contains. This reduces cognitive load when adding new API endpoints or debugging scenario failures.